CUDA-Accelerated FP-Growth Support-Threshold Study

Ten datasets, forty support settings, and CPU/CUDA performance analysis

Abstract

This project implements complete frequent-itemset mining using FP-Growth: a compressed frequent-pattern tree, linked headers, conditional pattern bases and recursive conditional trees. A separate CUDA build accelerates weighted support-counting histograms for the initial database and every recursive conditional tree. Tree construction and recursion remain on the CPU, making this a genuine **hybrid CUDA** implementation, not a fully GPU-resident tree miner.

Experiments evaluate ten distinct transaction datasets at four minimum-support thresholds each. Per-threshold measurements include exact frequent-itemset count, CPU time, host resident memory, and - where actual NVIDIA measurements exist - CUDA time, CUDA memory, kernel counts and CPU/CUDA speedup. Each dataset has its own four-panel performance figure and a results table.

Dataset provenance: four UCI-derived datasets and six reproducibly generated synthetic workloads. These are not ten UCI datasets. All measurements are from this project; none were imported from the comparison project.

Measured settings	Benchmark status	GPU status
10 x 4 = 40	CPU experiments completed	Real GPU runs recorded

Measured environment: aarch64; Linux-6.8.12-1021-tegra-aarch64-with-glibc2.39; GPU: Orin.

Timing: median of 3 independent runs per support level; C++ process timing including input parsing and mining, excluding Python startup and pattern CSV writing. Very small runs are sensitive to timer noise.

1. Problem definition and FP-Growth method

Given a transaction database D with N rows and an itemset X, support(X) is the number of transactions containing every item of X. At minimum support s, X is frequent if support(X) is at least ceil(sN). Our miner outputs every nonempty frequent itemset and its support, with duplicates removed inside each transaction.

1.1 Sequential miner

The initial scan counts item supports and discards infrequent tokens. Remaining items are sorted by decreasing support and inserted into an FP-tree. Shared prefixes share nodes. Header links connect nodes with the same item. For each suffix item, node-to-root ancestor paths form a weighted conditional pattern base. A conditional FP-tree is then built and recursively mined.

1.2 CUDA hybrid: actual GPU responsibilities

For both the root tree and recursively generated conditional pattern bases, CPU code flattens item IDs and path multiplicities into arrays. These arrays are copied to the NVIDIA device. A 64-bit atomicAdd histogram kernel computes weighted support values in parallel. The histogram returns to the CPU, which prunes and rebuilds the tree. This can accelerate large counting workloads but may be slower for many tiny conditional trees because of kernel-launch and transfer overhead.

```
CPU: generate weighted transactions from conditional pattern base
  -> flatten [item_id], [weight]
GPU: transfer arrays, reset support histogram, launch atomicAdd kernel
  -> transfer support histogram back to host
CPU: remove infrequent items, build conditional tree, recurse
```

1.3 Correctness approach

The weighted support of item j in a conditional base is the sum of path weights of all rows containing j. Each suffix ordering is explored once, so every frequent itemset is emitted once. The same input and threshold are evaluated with the CPU and actual CUDA executables. Their complete itemset/support CSVs are compared by scripts/verify_cuda.py, alongside small independent exhaustive checks. Benchmark summary equality alone is not a substitute for full pattern verification.

2. Manual validation and reproducibility

The five-basket toy example contains bread, milk, diaper, beer and eggs. With absolute minimum support 3, eight frequent itemsets are expected and were confirmed on the CPU reference: bread(4), milk(4), diaper(4), beer(3), bread+milk(3), bread+diaper(3), milk+diaper(3), and beer+diaper(3).

Additional CPU tests use 100 randomized small datasets checked against an independent subset-enumeration oracle. The CUDA host-simulator code path was tested separately, but a simulator is not proof of actual NVIDIA kernel execution. When performing lab benchmarks, run the strict itemset/support verifier before timing, and retain its terminal log.

2.1 Experiment design

Each of the ten input files is evaluated at four decreasing support thresholds. The thresholds are intentionally dataset-specific to obtain feasible yet informative pattern counts. The program records execution time including input parsing and mining, FP-tree metrics, patterns, maximum pattern length, process peak resident memory, and CUDA buffer high-water when available. Higher pattern counts generally increase work, but memory and runtime also depend on transaction length, unique items, tree compression and number of conditional-tree builds.

2.2 Avoiding misleading comparisons

CPU and CUDA times are compared only when measured on the same machine in the same script run; the plotted ratio is CPU median time / CUDA median time. A value greater than 1 means CUDA was faster, a value less than 1 means CPU was faster. GPU device buffer high-water excludes driver and context allocations and is NOT directly equivalent to peak GPU process memory. The program timer starts before input parsing and ends after mining; it excludes Python startup and bulk CSV pattern writing (--no-output), but includes CUDA launch/transfer overhead as measured inside the C++ binary.

Dataset selection: UCI Iris, UCI Wine, UCI Wisconsin Diagnostic Breast Cancer, UCI Handwritten Digits (all discretized as documented), plus synthetic Grocery, Bundles, Clickstream, Sensors, Long-tail, and Retail 50k. The six synthetic files are for stress testing and are not described as UCI benchmarks.

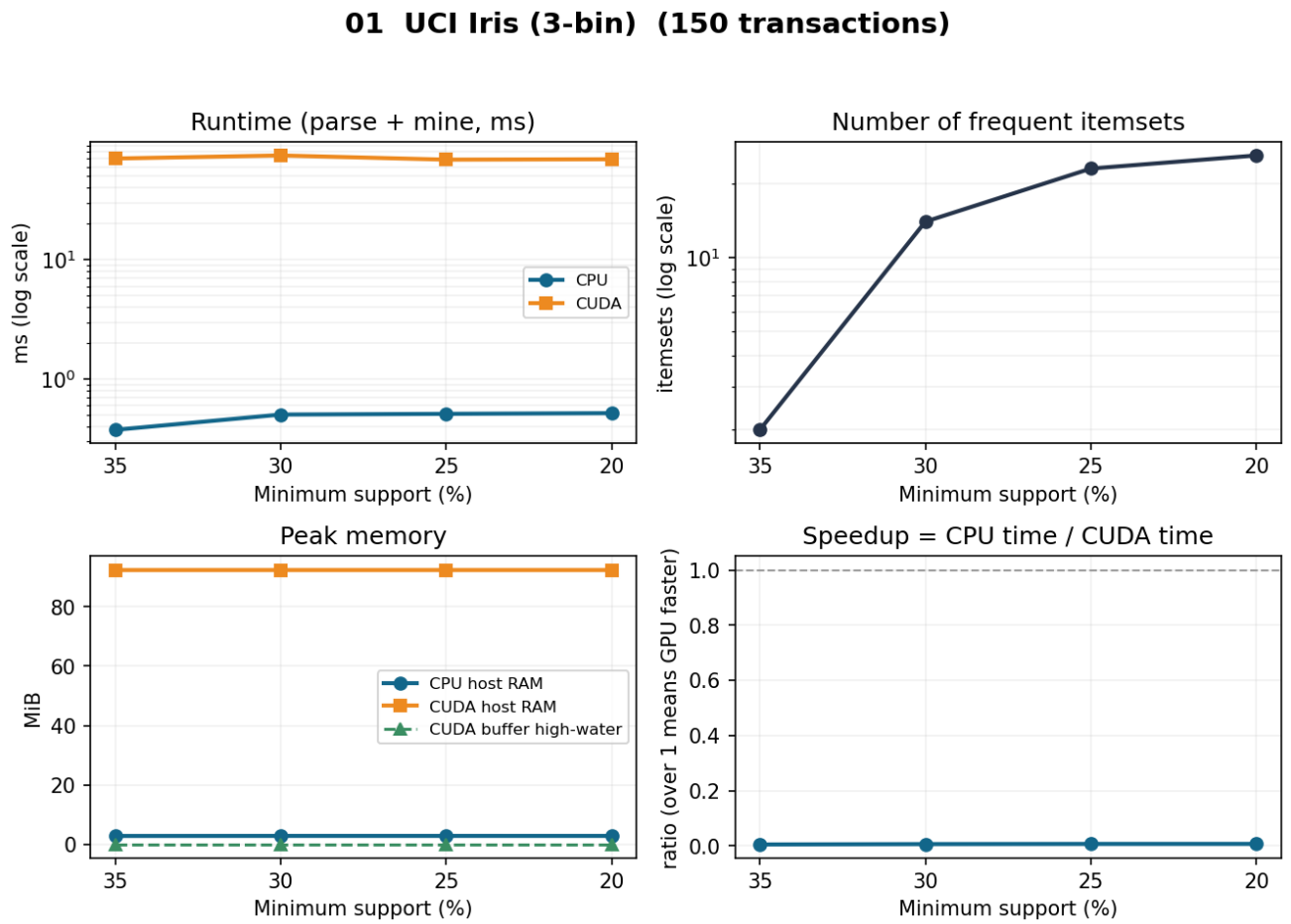
3. The ten datasets and experimental settings

# / dataset	Origin	Rows	Support sweep	Patterns at lowest
1. Iris	UCI-derived	150	35% to 20%	26
2. Wine	UCI-derived	178	50% to 30%	153
3. WDBC	UCI-derived	569	50% to 35%	1,684
4. Digits	UCI-derived	1797	90% to 75%	1,228
5. Grocery	Synthetic	3000	20% to 10%	143
6. Bundles	Synthetic	6000	20% to 12%	60
7. Clickstream	Synthetic	12000	4% to 1.5%	71
8. Sensors	Synthetic	8000	45% to 30%	32
9. Long-tail	Synthetic	20000	10% to 4%	127
10. Retail 50k	Synthetic	50000	10% to 4%	164

Every file above is part of the project ZIP. Data origins, transformations and reproducibility notes are provided in data/TEN_DATASET_SOURCES.md. The support settings are fixed in scripts/benchmark_sweep.py so repeating the experiment does not require editing the report.

What is shown next: each dataset receives a separate page with CPU runtime and CUDA runtime (when measured), number of frequent itemsets, memory, and speedup against minimum support; a table provides the exact measurements behind every plotted point.

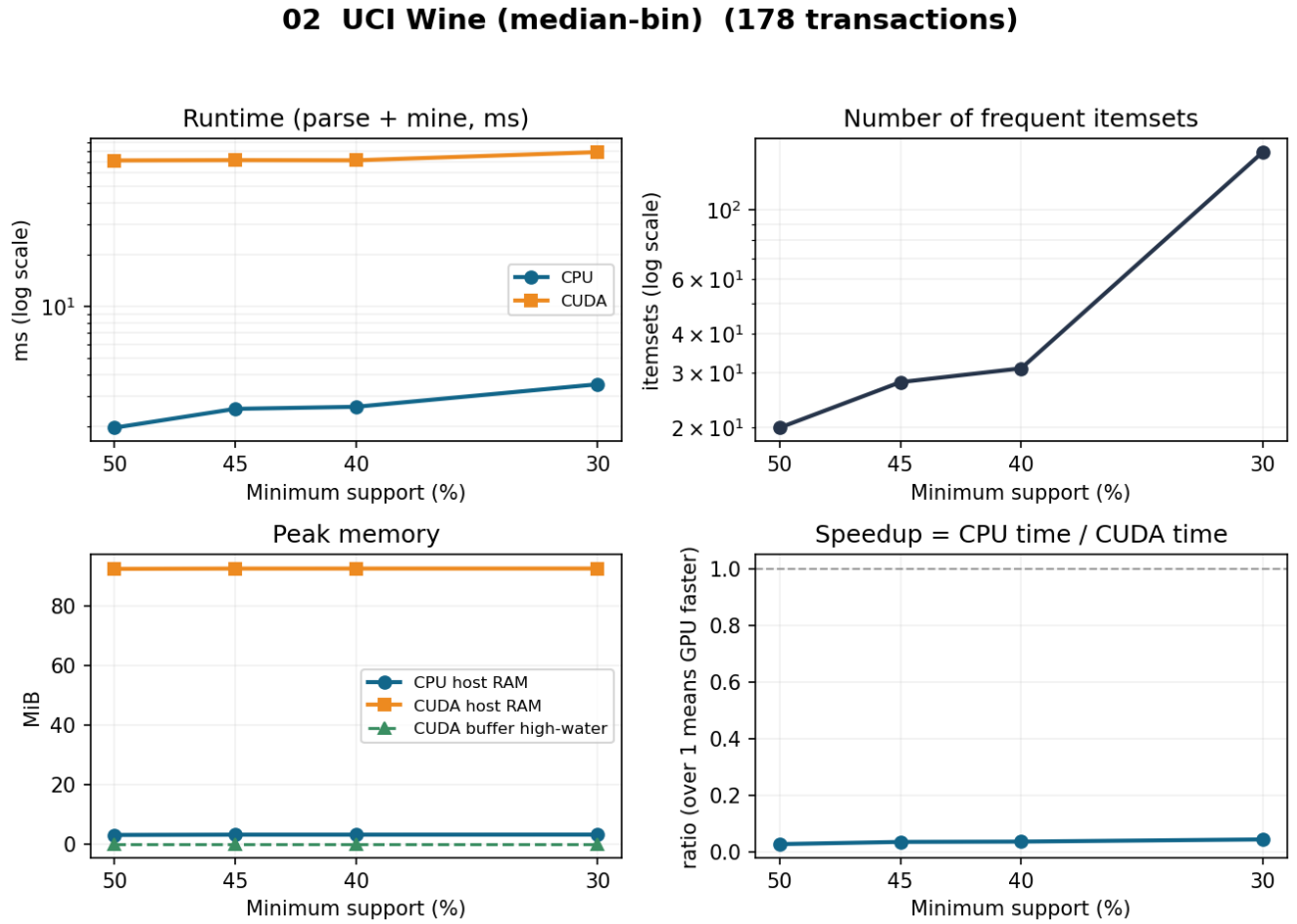
4. Dataset analysis: UCI Iris (3-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
35%	2	0.376	70.255	2.87	0.01x
30%	14	0.505	74.554	2.89	0.01x
25%	23	0.512	68.742	2.89	0.01x
20%	26	0.519	69.289	2.88	0.01x

Lowering support from 35% to 20% changed the frequent itemset count from 2 to 26. At the lowest support, the CPU median runtime was 0.52 ms and peak host RSS was 2.88 MiB. CUDA median time was 69.29 ms, corresponding to 0.01x CPU/CUDA speedup (GPU slower).

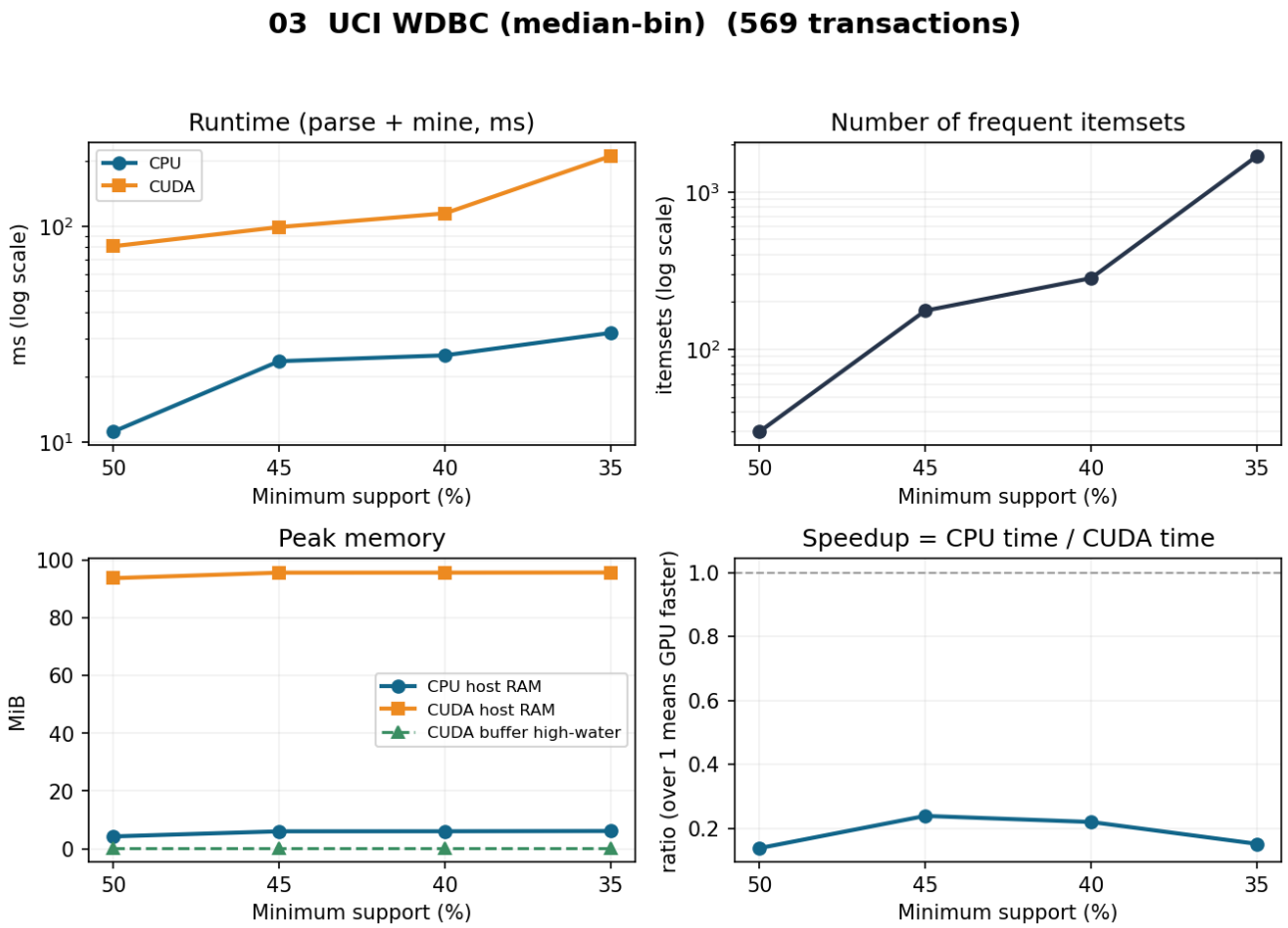
5. Dataset analysis: UCI Wine (median-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
50%	20	1.964	71.008	3.11	0.03x
45%	28	2.530	71.297	3.21	0.04x
40%	31	2.601	71.109	3.21	0.04x
30%	153	3.519	79.401	3.23	0.04x

Lowering support from 50% to 30% changed the frequent itemset count from 20 to 153. At the lowest support, the CPU median runtime was 3.52 ms and peak host RSS was 3.23 MiB. CUDA median time was 79.40 ms, corresponding to 0.04x CPU/CUDA speedup (GPU slower).

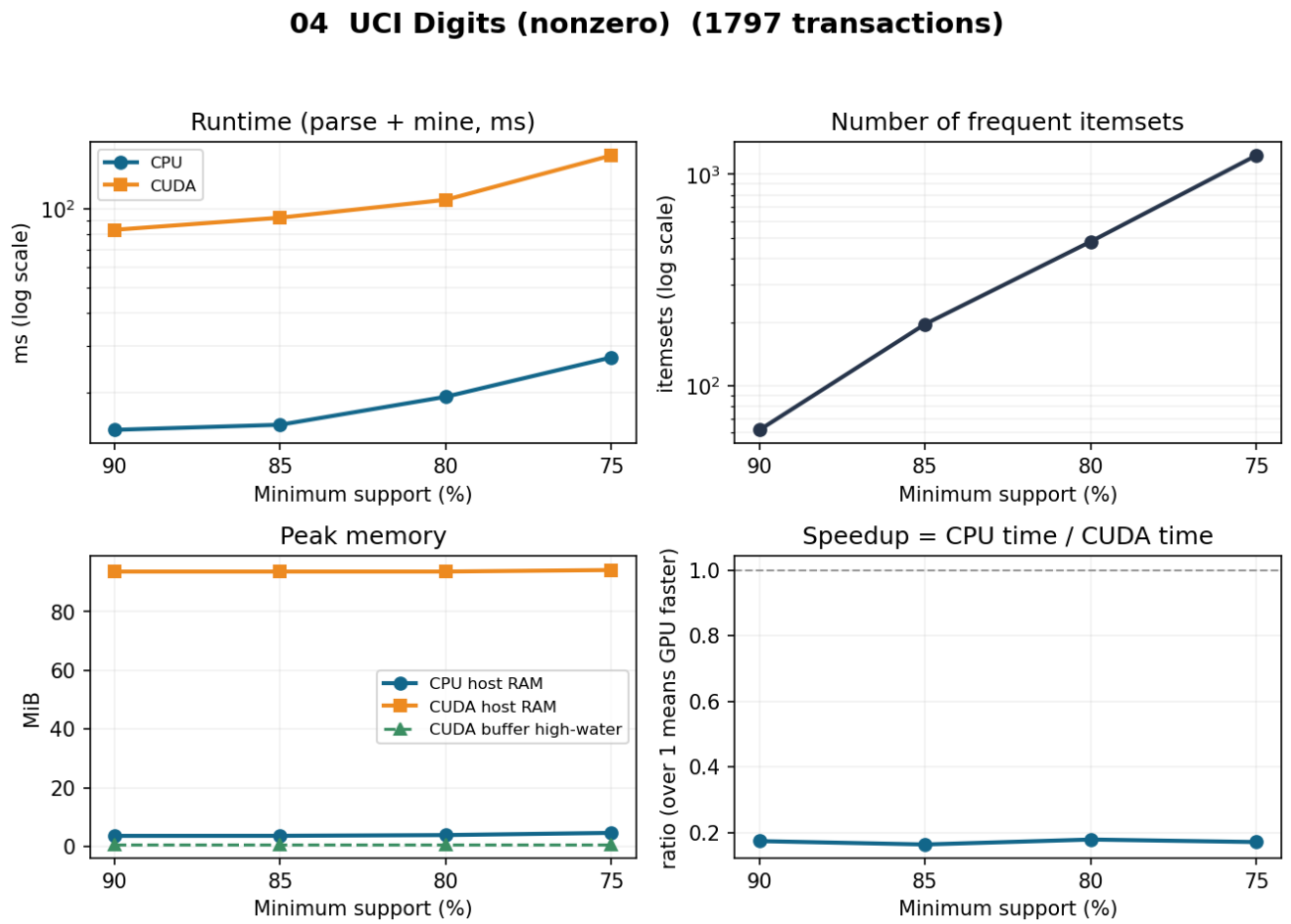
6. Dataset analysis: UCI WDBC (median-bin)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
50%	30	11.120	80.658	4.25	0.14x
45%	176	23.618	98.990	6.02	0.24x
40%	283	25.148	114.524	6.04	0.22x
35%	1,684	31.958	211.231	6.12	0.15x

Lowering support from 50% to 35% changed the frequent itemset count from 30 to 1,684. At the lowest support, the CPU median runtime was 31.96 ms and peak host RSS was 6.12 MiB. CUDA median time was 211.23 ms, corresponding to 0.15x CPU/CUDA speedup (GPU slower).

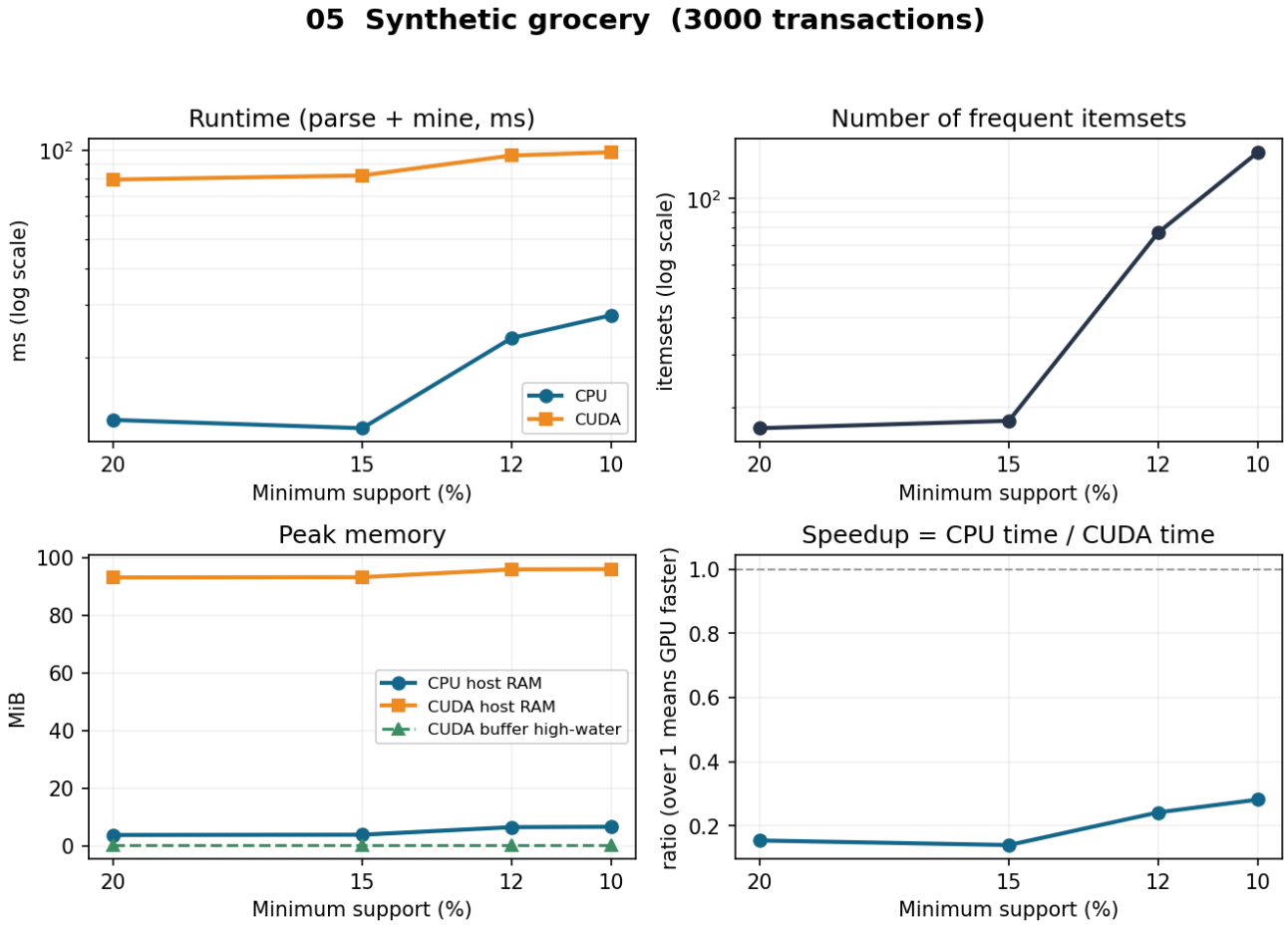
7. Dataset analysis: UCI Digits (nonzero)



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
90%	62	14.419	83.021	3.60	0.17x
85%	196	15.080	92.309	3.62	0.16x
80%	481	19.262	107.928	3.88	0.18x
75%	1,228	27.184	159.129	4.62	0.17x

Lowering support from 90% to 75% changed the frequent itemset count from 62 to 1,228. At the lowest support, the CPU median runtime was 27.18 ms and peak host RSS was 4.62 MiB. CUDA median time was 159.13 ms, corresponding to 0.17x CPU/CUDA speedup (GPU slower).

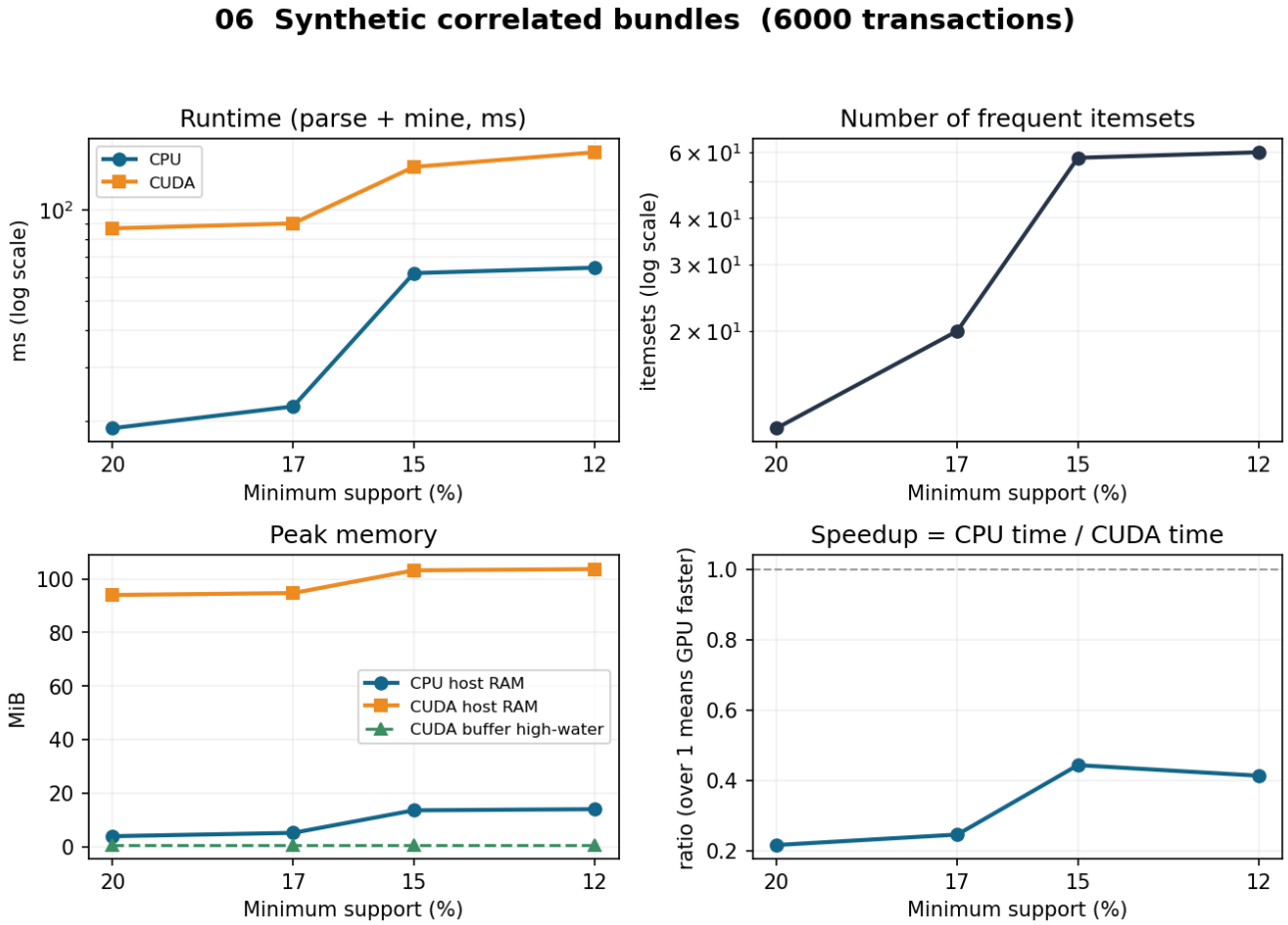
8. Dataset analysis: Synthetic grocery



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
20%	17	12.297	79.656	3.79	0.15x
15%	18	11.532	82.232	3.88	0.14x
12%	77	23.246	96.056	6.50	0.24x
10%	143	27.750	98.471	6.62	0.28x

Lowering support from 20% to 10% changed the frequent itemset count from 17 to 143. At the lowest support, the CPU median runtime was 27.75 ms and peak host RSS was 6.62 MiB. CUDA median time was 98.47 ms, corresponding to 0.28x CPU/CUDA speedup (GPU slower).

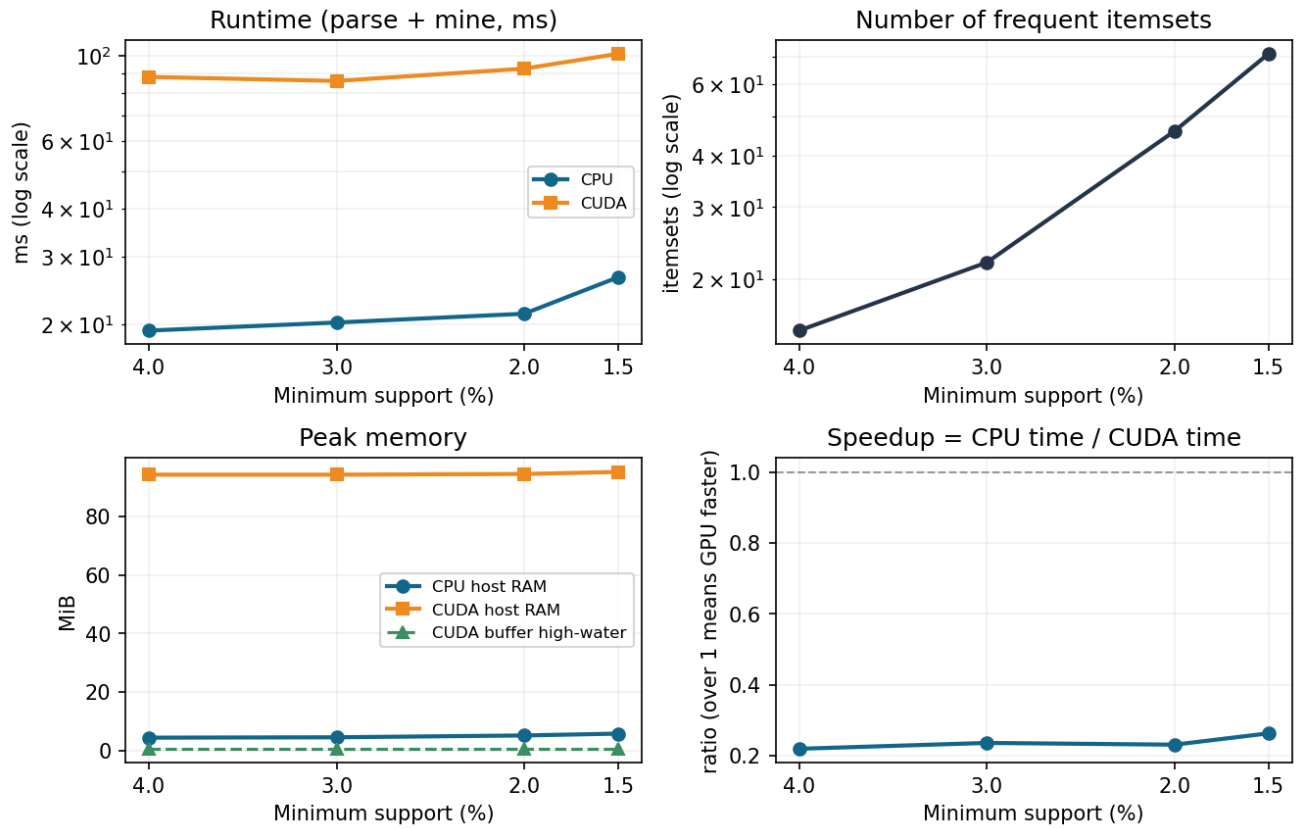
9. Dataset analysis: Synthetic correlated bundles



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
20%	11	18.861	87.009	4.00	0.22x
17%	20	22.284	90.384	5.25	0.25x
15%	58	61.785	139.283	13.66	0.44x
12%	60	64.398	155.740	14.07	0.41x

Lowering support from 20% to 12% changed the frequent itemset count from 11 to 60. At the lowest support, the CPU median runtime was 64.40 ms and peak host RSS was 14.07 MiB. CUDA median time was 155.74 ms, corresponding to 0.41x CPU/CUDA speedup (GPU slower).

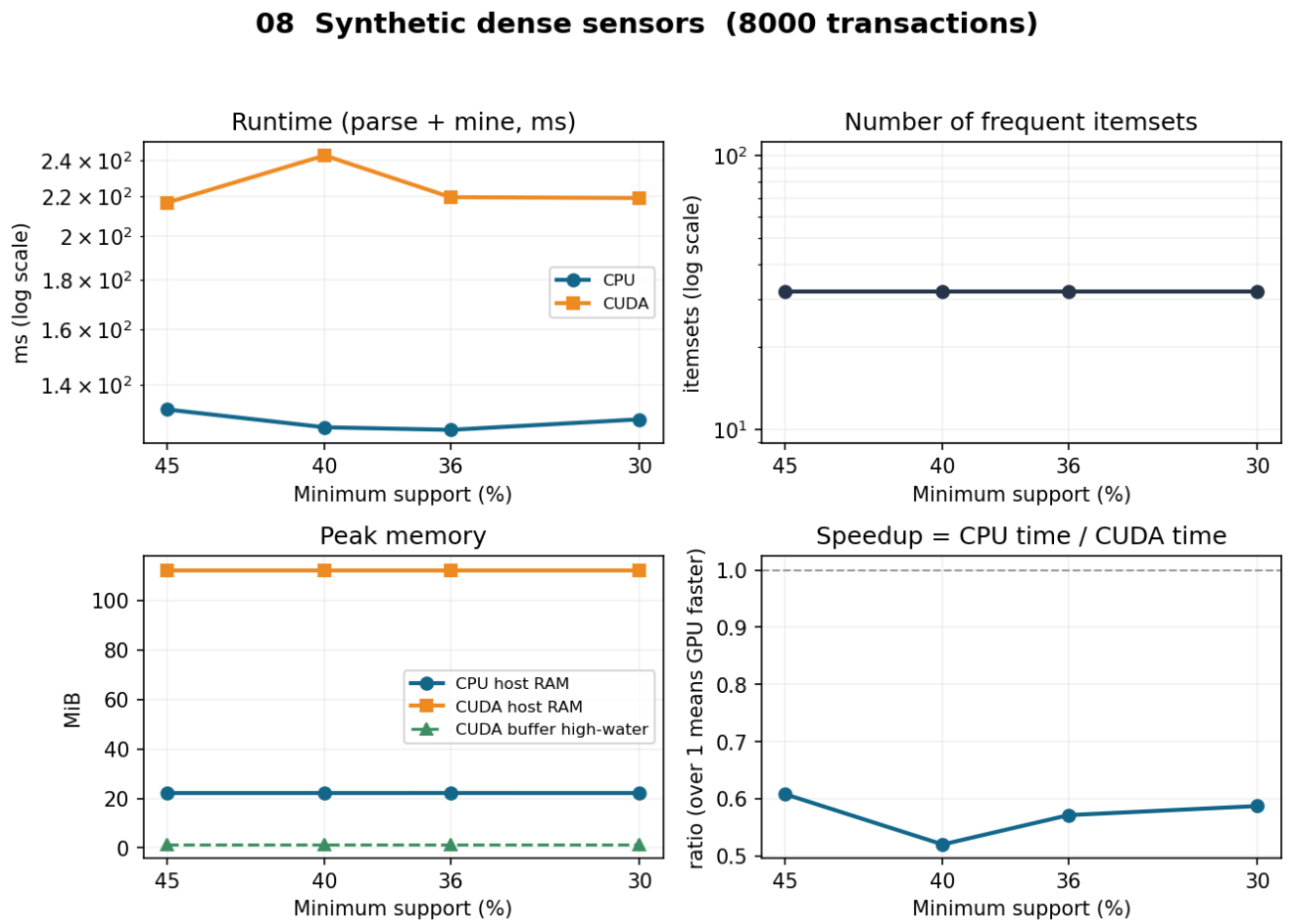
10. Dataset analysis: Synthetic sparse clickstream



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
4%	15	19.301	88.122	4.35	0.22x
3%	22	20.257	86.023	4.47	0.24x
2%	46	21.353	92.599	5.10	0.23x
1.5%	71	26.569	101.205	5.72	0.26x

Lowering support from 4% to 1.5% changed the frequent itemset count from 15 to 71. At the lowest support, the CPU median runtime was 26.57 ms and peak host RSS was 5.72 MiB. CUDA median time was 101.20 ms, corresponding to 0.26x CPU/CUDA speedup (GPU slower).

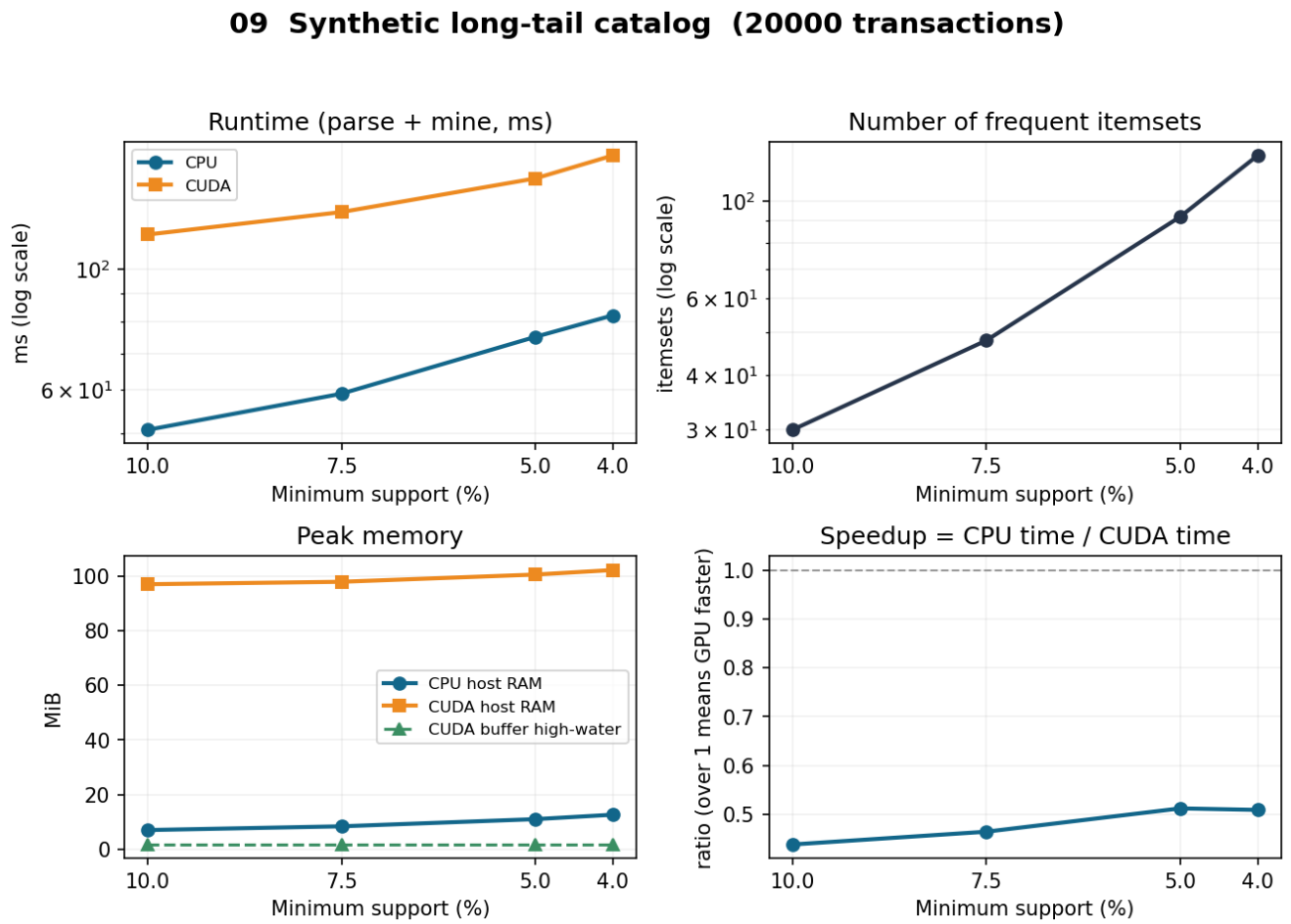
11. Dataset analysis: Synthetic dense sensors



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
45%	32	131.948	216.867	22.17	0.61x
40%	32	126.441	242.884	22.17	0.52x
36%	32	125.657	219.708	22.17	0.57x
30%	32	128.878	219.283	22.17	0.59x

Lowering support from 45% to 30% changed the frequent itemset count from 32 to 32. At the lowest support, the CPU median runtime was 128.88 ms and peak host RSS was 22.17 MiB. CUDA median time was 219.28 ms, corresponding to 0.59x CPU/CUDA speedup (GPU slower).

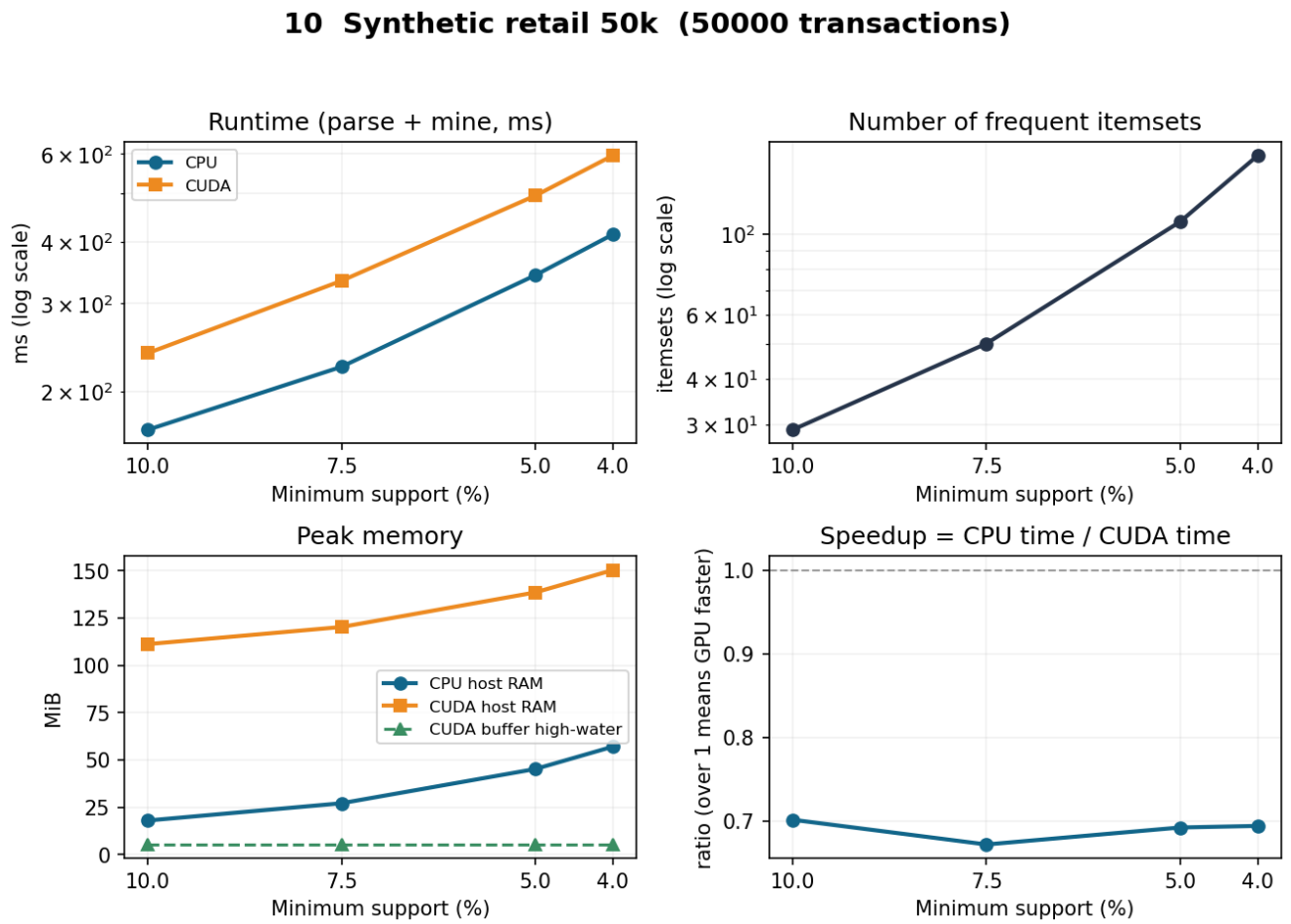
12. Dataset analysis: Synthetic long-tail catalog



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
10%	30	50.675	115.545	7.00	0.44x
7.5%	48	59.029	127.023	8.38	0.46x
5%	92	75.024	146.414	11.00	0.51x
4%	127	82.174	161.335	12.63	0.51x

Lowering support from 10% to 4% changed the frequent itemset count from 30 to 127. At the lowest support, the CPU median runtime was 82.17 ms and peak host RSS was 12.63 MiB. CUDA median time was 161.34 ms, corresponding to 0.51x CPU/CUDA speedup (GPU slower).

13. Dataset analysis: Synthetic retail 50k



Min sup	Itemsets	CPU ms	CUDA ms	CPU MiB	Speedup
10%	29	167.233	238.474	17.96	0.70x
7.5%	50	223.959	333.426	27.09	0.67x
5%	108	342.242	494.572	45.20	0.69x
4%	164	413.385	595.739	57.03	0.69x

Lowering support from 10% to 4% changed the frequent itemset count from 29 to 164. At the lowest support, the CPU median runtime was 413.38 ms and peak host RSS was 57.03 MiB. CUDA median time was 595.74 ms, corresponding to 0.69x CPU/CUDA speedup (GPU slower).

14. Discussion and conclusions

On the prepared datasets, decreasing the support threshold increases or preserves the number of frequent itemsets, as required mathematically. Runtime does not have to increase monotonically because of measurement noise and differences in tree traversal, but for most datasets a lower threshold makes more itemsets and potentially more conditional trees. The sensitivity to support is why a single support setting cannot characterize FP-Growth performance.

This implementation retains FP-tree construction, recursive traversal and conditional-base management on the CPU, so GPU acceleration is confined to repeated weighted histograms. It differs materially from a GPU-resident FP-Growth design. Many small CUDA launches can dominate end-to-end execution even if histogram operations parallelize well. Conclusions about GPU speedup should be based on the 40 actual same-machine measurements, not performance figures from another implementation or GPU.

Limitations and future work

The prepared ten-dataset bundle comprises four UCI-derived files and six synthetic workloads. A requirement for ten original UCI datasets would need six replacements before submission. The chosen thresholds are not uniform and were selected for feasible runtime, so cross-dataset timing should be interpreted within each dataset rather than as a controlled comparison of difficulty. Small datasets have timing noise. A fully GPU-resident implementation or conditional-pattern-base batching could change the performance balance, but neither is claimed as implemented.

References

Han, J., Pei, J. and Yin, Y. (2000). Mining Frequent Patterns without Candidate Generation. ACM SIGMOD. https://doi.org/10.1145/342009.335372

UCI Machine Learning Repository. https://archive.ics.uci.edu/

NVIDIA CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/

15. Lab commands and submission checklist

```
make
make cuda
python3 scripts/verify_cuda.py --random-cases 20
python3 scripts/benchmark_sweep.py --with-cuda --repeats 3
python3 scripts/generate_sweep_report.py
```

Install PDF dependencies through Ubuntu system packages if they are not available: sudo apt install python3-reportlab python3-matplotlib. Do not use sudo pip, especially on a shared lab workstation. Alternatively create a Python virtual environment if permitted.

Save the strict comparison output as a log, verify that the report explicitly names the NVIDIA GPU and prints CPU/GPU speedup for all forty settings, inspect all ten charts, and only then print the PDF and email your own source code, datasets and report in the ZIP. Ensure the assignment due date is confirmed with the instructor.

Source note: This report is generated from results/support_sweep_results.csv created by this project. The comparison project supplied by a classmate was used to identify useful experimental presentation practices, not as a source for benchmark numbers or as code to copy.